

Unidad II - Etapa 2: Entregable
Diseño conceptual y lógico de la base de datos del proyecto

Carlos Alfonso Muñoz Agudelo
Esteban Giovanni Garay Cano
Carlos Sebastian Castillo Silva

Docente: Miguel Alfonso Varela

Universidad de la Sabana
Facultad de Ingeniería
Maestría en Arquitectura de Software
Diseño y Optimización de Bases de Datos
2026

Índice

Introducción	4
Abstract	4
Objetivos	5
Objetivo general	5
Objetivos específicos	5
Resumen ejecutivo	5
Información del equipo	6
Enfoque Arquitectónico Seleccionado	7
Arquitectura Políglota Híbrida	7
Módulo Transaccional en Postgresql	8
Script para creación de Bases de Datos Postgresql	8
Módulo NoSQL para MongoDB	9
Diseño preliminar del módulo NoSQL para MongoDB.	9
MongoDB Atlas	10
Script para creación de Bases de Datos Mongodb	10
Abrir MongoDB Compass	10
Análisis de requisitos	11
Contexto de Ecommify	11
Requisitos funcionales	12
Módulo Transaccional y Logístico - Postgresql	12
Módulo de Catálogo y Experiencia de Usuario - MongoDB	12
Requisitos no funcionales	13
Identificación de entidades del dominio	14
Entidades del Dominio Relacional (PostgreSQL)	14
Entidades del Dominio Documental (MongoDB)	15
Diseño conceptual	16
Diagrama ER	17
Diseño SQL para PostgreSQL.	17
Diseño NoSQL para MongoDB	18
Descripción Detallada de las Entidades en PostgreSQL	18
Descripción Detallada de las Entidades en MongoDB	20
Descripción Detallada de las Colecciones	20
Colección de Catálogo de Productos	20
Colección de Sesiones de Clientes	21
Descripción de Relaciones e Integración Híbrida	21
Relaciones por Incrustación (Relación 1:1 y 1:N Interna)	21
Relación Lógica Inter-Motor (Acoplamiento Débil con PostgreSQL)	21
Restricciones de negocio identificadas	22
Diseño lógico - Módulo PostgreSQL	23
Diseño lógico preliminar - Módulo MongoDB.	27
Colecciones principales	27

Estructura propuesta del documento	27
customer_sessions_collection	29
Patrones de modelado aplicados	30
1. Patrón de incrustación	30
2. Patrón de documento autocontenido	30
3. Patrón de referencia lógica entre motores	30
4. Patrón de resumen precalculado	30
5. Patrón TTL para sesiones temporales	30
Índices recomendados en MongoDB	30
Clasificación de qué datos van a MongoDB vs PostgreSQL	31
Decisiones arquitectónicas justificadas	31
Usar PostgreSQL para el núcleo transaccional	31
Usar MongoDB para catálogo y experiencia de usuario	32
Usar acoplamiento débil entre PostgreSQL y MongoDB	32
Usar documentos embebidos en MongoDB	32
Usar resumen precalculado de reseñas	33
EDA del dataset Olist	33
Relaciones entre tablas	34
Calidad de datos	35
Estadísticas básicas	35
Distribución geográfica	36
Distribución temporal	37
Valores nulos	37
Outliers	38
Patrones del dataset	38
Matriz de decisiones	39
Análisis del Teorema CAP aplicado	41
PostgreSQL: enfoque CP para el núcleo transaccional	41
MongoDB: enfoque AP para catálogo, sesiones y carritos	41
Aplicación en Ecommify	42
Trade-offs y estrategias de mitigación	42
Flujos de sincronización entre PostgreSQL y MongoDB	44
Flujo 1: Creación de producto	44
Flujo 2: Actualización de catálogo	44
Flujo 3: Navegación del usuario	44
Flujo 4: Carrito de compras	44
Flujo 5: Checkout y confirmación de compra	45
Anexos	45
Anexo A. Diccionario de datos preliminar PostgreSQL	45
Anexo B. Diccionario de datos preliminar MongoDB	47
Anexo C. Script preliminar MongoDB	48
Anexo D. Ejemplo de documento de producto en MongoDB	49
Anexo E. Ejemplo de documento de sesión y carrito	49
Presentación ejecutiva	50

Repositorio GitHub con código preliminar	50
Referencias	51

50
51

Introducción

El presente documento desarrolla el diseño conceptual y lógico de la base de datos para el proyecto Ecommify, una plataforma de comercio electrónico multiproveedor inspirada en el ecosistema de datos de Olist, debido a la naturaleza del negocio, el sistema requiere manejar información transaccional crítica, datos logísticos, catálogos de productos, promociones, sesiones de usuarios y carritos de compra activo, dada esta razón, se propone una arquitectura de persistencia políglota híbrida, en la cual cada tipo de información se almacena en el motor de base de datos que mejor responde a sus necesidades de consistencia, rendimiento y flexibilidad.

El diseño plantea el uso de PostgreSQL para el núcleo transaccional, financiero y logístico, debido a su soporte de propiedades ACID, integridad referencial, tipos avanzados, extensiones como PostGIS y capacidad para modelar datos estructurados de manera robusta. Por otro lado, se propone MongoDB para el módulo de catálogo, promociones, reseñas resumidas, sesiones y carritos temporales, aprovechando su modelo documental, flexible y orientado a lecturas rápidas.

Abstract

El presente trabajo desarrolla el diseño conceptual y lógico de la base de datos para Ecommify, una plataforma de comercio electrónico multiproveedor basada en el análisis del dataset Olist. La propuesta se fundamenta en una arquitectura políglota híbrida, integrando PostgreSQL y MongoDB según las necesidades específicas de cada módulo del sistema.

PostgreSQL se selecciona para el núcleo transaccional, financiero y logístico, debido a su soporte de propiedades ACID, integridad referencial, precisión en datos monetarios, tipos avanzados como JSONB, arreglos, tipos compuestos y rangos temporales, además de extensiones como PostGIS para el análisis geoespacial, por otra parte, MongoDB se utiliza para el catálogo de productos, promociones, sesiones de usuario y carritos de compra, aprovechando su modelo documental flexible, su capacidad de incrustar datos relacionados y su orientación a consultas de baja latencia.

A partir del análisis exploratorio de datos se identificaron relaciones clave entre clientes, órdenes, productos, vendedores, pagos y geolocalización, lo que permitió justificar la separación entre datos transaccionales y datos orientados a experiencia de usuario, para finalizar, se propone una matriz de decisiones, un análisis del Teorema CAP y flujos de sincronización entre motores, con el fin de garantizar consistencia en operaciones críticas y disponibilidad en procesos de navegación masiva.

Objetivos

Objetivo general

Diseñar una arquitectura de base de datos híbrida para el proyecto Ecommify, integrando PostgreSQL y MongoDB con el fin de responder de manera eficiente a las necesidades transaccionales, logísticas, documentales y de experiencia de usuario propias de una plataforma de comercio electrónico multiproveedor.

Objetivos específicos

1. Identificar los requisitos funcionales y no funcionales de Ecommify, diferenciando las necesidades del módulo transaccional, logístico, documental y de experiencia de usuario.
2. Definir las entidades principales del dominio, separando aquellas que requieren consistencia estricta en PostgreSQL de aquellas que necesitan flexibilidad y alta disponibilidad en MongoDB.
3. Diseñar el modelo lógico preliminar del módulo MongoDB, incluyendo colecciones, estructura de documentos, patrones de modelado y justificación técnica.
4. Aplicar el análisis del Teorema CAP para explicar los compromisos entre consistencia, disponibilidad y tolerancia a particiones en los módulos PostgreSQL y MongoDB.
5. Construir una matriz de decisión que evidencie qué datos deben almacenarse en PostgreSQL y cuáles en MongoDB, de acuerdo con criterios técnicos y de negocio.
6. Proponer una estructura preliminar de repositorio GitHub que organice scripts SQL, esquemas MongoDB, documentación técnica, notebooks de análisis exploratorio y anexos del proyecto.

Resumen ejecutivo

Ecommify es una plataforma de comercio electrónico multiproveedor que debe atender operaciones de compra, pagos, gestión logística, consulta de productos, promociones, sesiones de usuario y carritos de compra activos. Debido a esta variedad de necesidades, se propone una arquitectura de persistencia políglota híbrida que combina PostgreSQL y MongoDB.

PostgreSQL se selecciona como motor principal para el módulo transaccional y logístico, ya que permite garantizar consistencia, integridad referencial y precisión financiera. Este motor almacena clientes, vendedores, órdenes, detalles de órdenes, pagos y datos de geolocalización. Además, permite aprovechar tipos avanzados como `JSONB`, `TSTZRANGE`, arreglos, tipos compuestos y extensiones como PostGIS para cálculos de distancia entre vendedores y clientes.

MongoDB se utiliza para el módulo de catálogo y experiencia de usuario, donde se priorizan lecturas rápidas, disponibilidad y flexibilidad de esquema, este motor almacena documentos de productos con promociones activas, atributos variables, galerías multimedia y resúmenes de reseñas. También gestiona sesiones de clientes y carritos temporales mediante documentos embebidos que permiten recuperar la información en una sola lectura.

La decisión arquitectónica se fundamenta en separar las responsabilidades de cada motor. PostgreSQL protege las operaciones críticas del negocio bajo propiedades ACID, mientras que MongoDB permite escalar la navegación del catálogo y mantener baja latencia durante periodos de alta concurrencia. En conjunto, la solución permite construir una base de datos más flexible, escalable y alineada con las necesidades reales de una plataforma de comercio electrónico moderna.

Información del equipo

El equipo está conformado por:

Integrante	Rol dentro del proyecto	Responsabilidades principales
Carlos Alfonso Muñoz Agudelo	Diseño transaccional y analítico	Diseño del módulo PostgreSQL, normalización, modelo relacional, particionamiento y consultas analíticas.
Esteban Giovanni Garay Cano	Apoyo técnico y configuración	Apoyo en configuración de entornos, conexión con servicios cloud, MongoDB Atlas y validación de scripts.
Carlos Sebastian Castillo Silva	Diseño híbrido, MongoDB y decisiones arquitectónicas	Desarrollo de introducción, abstract, objetivos, resumen ejecutivo, diseño lógico preliminar MongoDB, matriz de decisiones, análisis CAP, anexos y estructura de repositorio.

La distribución del trabajo permite abordar el diseño desde diferentes perspectivas: consistencia transaccional, flexibilidad documental, análisis de datos y justificación arquitectónica. Esta división también facilita que cada componente del sistema sea desarrollado con criterios técnicos específicos y alineados con los objetivos del proyecto.

Enfoque Arquitectónico Seleccionado

Arquitectura Políglota Híbrida

El enfoque arquitectónico seleccionado es la Arquitectura Políglota Híbrida.

El sistema Ecommify adopta una Arquitectura Políglota Híbrida. Este paradigma de diseño establece que una plataforma de software moderna no debe limitarse a un único tipo de motor de base de datos, sino que debe fragmentar su modelo de dominio para almacenar cada tipo de dato en el motor que mejor se adapte a sus requisitos operativos, de rendimiento y de consistencia.

La selección de una Arquitectura Políglota Híbrida para Ecommify se fundamenta en tres pilares arquitectónicos:

1. Separación de Responsabilidades según el Teorema CAP

- **PostgreSQL:** Se selecciona para el core del negocio financiero. En este módulo es inaceptable la pérdida de datos; se requiere consistencia inmediata que garantice que cualquier transacción sólo llevará a la base de datos a un estado válido, respetando todas las reglas, restricciones y esquemas predefinidos esto para evitar pérdidas de dinero, duplicación de cobros o descuadres de inventario durante el procesamiento de pagos.
En PostgreSQL, las propiedades ACID se implementan de forma automática e interna mediante su arquitectura central, por lo que el desarrollador solo necesita gestionar correctamente los bloques de las transacciones
- **MongoDB:** Se delega para los módulos interactivos. En el catálogo y el carrito de compras se priorizan las lecturas masivas veloces y la alta disponibilidad global. Si un servidor experimenta retrasos, es preferible mostrar una promoción con una ligera latencia de actualización a que la página web deje de responder por completo.

2. Optimización del Modelo de Datos

- Las entidades operacionales como Clientes, Vendedores y la infraestructura logística de Geolocalización espacial (PostGIS) poseen atributos estructurados y estables en el tiempo, ideales para el modelo tabular relacional de Postgres.
- Por el contrario, las especificaciones técnicas de un producto son altamente heterogéneas, por ejemplo, un electrodoméstico requiere campos de voltaje; una prenda de ropa requiere campos de talla y color. Almacenar esto en SQL obligaría a tener tablas llenas de valores nulos o diseños complejos. MongoDB resuelve esto nativamente permitiendo polimorfismo y almacenamiento dinámico de objetos incrustados.

3. Mitigación del Costo Computacional

- En el módulo transaccional, la normalización es obligatoria para garantizar la integridad referencial a través de llaves foráneas, evitando que existan ítems huérfanos sin una orden de compra válida.
- En el portal de navegación del cliente, realizar múltiples operaciones de unión entre tablas de productos, fotos, promociones y resúmenes de reseñas degradaría

drásticamente los tiempos de carga en momentos de alta demanda como un Black Friday. MongoDB mitiga este costo mediante el patrón de incrustación, permitiendo que la ficha de un producto se recupere en una sola operación de lectura.

Módulo Transaccional en Postgresql

Diseño completo conceptual y lógico del módulo transaccional de Ecommify que será implementado en PostgreSQL

El núcleo transaccional opera bajo el estándar ACID para garantizar la consistencia absoluta en el dinero, el inventario y los estados logísticos.

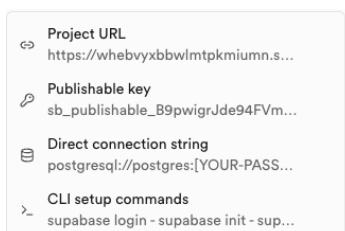
- Clientes y Vendedores (Entidades Fuertes): Poseen identidades únicas pero comparten una estructura semántica idéntica de localización, modelada mediante abstracciones de tipos de datos.
- Productos (Entidad Objeto-Relacional): Rompe el esquema relacional rígido al encapsular las especificaciones variables y los activos multimedia directamente en su registro raíz.
- Órdenes e Ítems (Relación Maestro-Detalle): El ciclo de vida temporal se modela de forma continua, mientras que los flujos financieros de pago se encapsulan jerárquicamente para mitigar la sobrecarga de uniones (JOINS).

Script para creación de Bases de Datos Postgresql

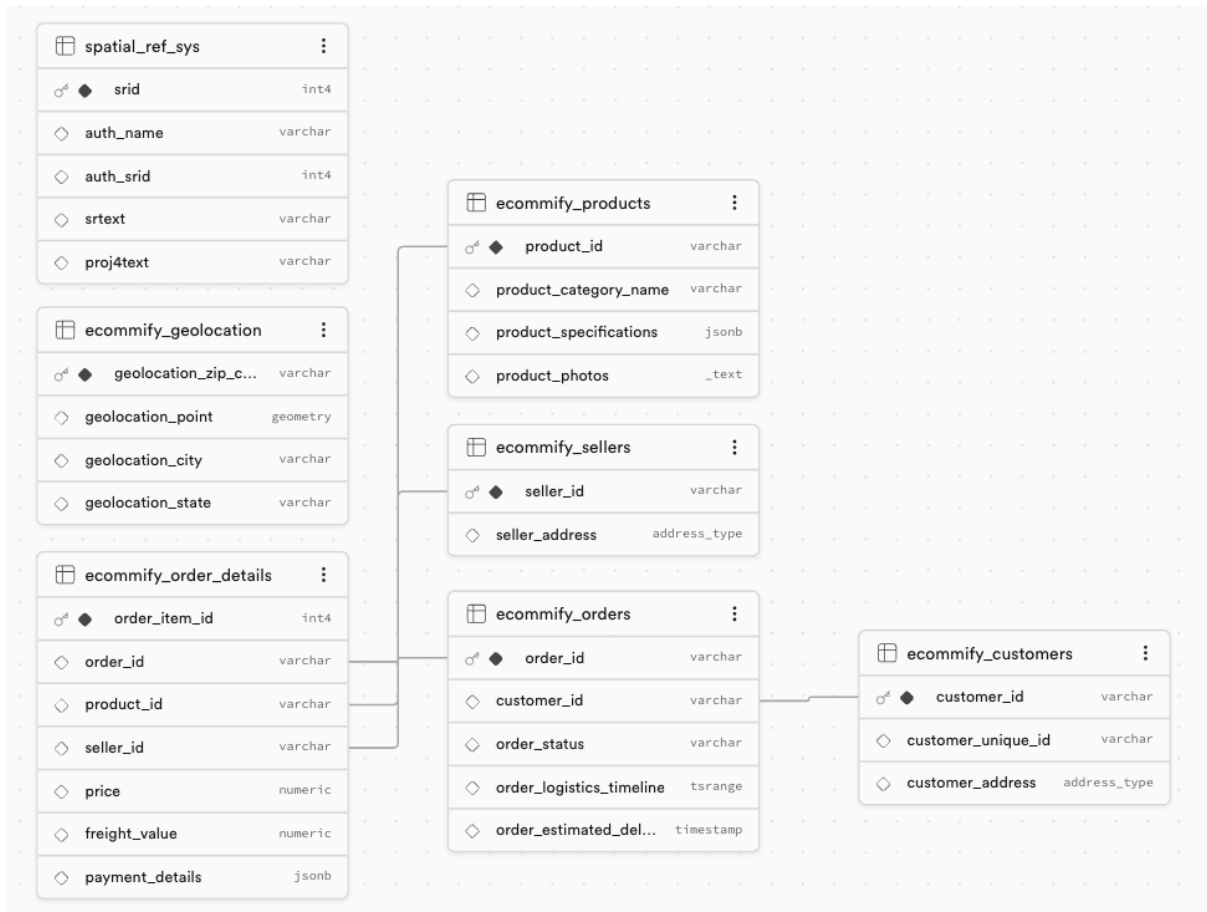
[ecommmify_db.sql](#)

<https://supabase.com/>

Password: Postgres123+



- <https://whebvxybbwlmtpkmiurn.supabase.co>
- postgresql://postgres:[YOUR-PASSWORD]@[db.whebvxybbwlmtpkmiurn.supabase.co:5432/postgres](https://whebvxybbwlmtpkmiurn.supabase.co:5432/postgres)
- supabase login
supabase init
supabase link --project-ref whebvxybbwlmtpkmiurn



Módulo NoSQL para MongoDB

Diseño preliminar del módulo NoSQL para MongoDB.

Mientras PostgreSQL protege la integridad financiera, MongoDB asume el control del catálogo web masivo y de los sistemas de personalización. Su arquitectura orientada a documentos es ideal para lecturas ultrarrápidas con *queries de baja latencia* al evitar por completo operaciones de unión de datos.

CATALOG_PRODUCTS_COLLECTION			
string	_id	PK	
string	category		
object	active_promotions		Incrustado: is_promo, discount, valid_until
object	attributes		Incrustado: weight, dimensions(L, H, W)
array	media		Lista: [url_p0, url_p1, ...]
object	reviews_summary		Incrustado: avg_score, total_reviews

CUSTOMER_SESSIONS_COLLECTION			
string	_id	PK	session_uuid
string	customer_id		
timestamp	last_activity		
object	active_cart		Incrustado: items[prod_id, qty, added_at], total

MongoDB Atlas

[MongoDB Atlas Proyecto_Olist_Kaggle](#)

Script para creación de Bases de Datos Mongodb

Database Name: DataBaseOlistKaggle

Collection Name: CollectionBaseOlistKaggle

[ecommmify_nosql_db.sql](#)

Abrir MongoDB Compass

Descargar MongoDB Compass

<https://downloads.mongodb.com/compass/mongodb-compass-1.49.6-darwin-arm64.dmg>

Usar esta cadena de conexión en el aplicativo MongoDB Compass

```
mongodb+srv://estebangaraycano_db_user:zkePtPQm1Kw8W4fl@clusterolistkaggle.yjfzm0i.mongodb.net/
```

New Connection

Manage your connection settings

URI ⓘ

Edit Connection String 

mongodb+srv://estebangaraycano_db_user:*****@clusterolistkaggle.yjfzm0i.mongodb.net/

Name

clusterolistkaggle.yjfzm0i.mongodb.net

Color

No Color



Favorite this connection

Favoriting a connection will pin it to the top of your list of connections

➤ Advanced Connection Options

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.

[See example](#)

How do I format my connection string?

[See example](#)

Cancel

Save

Connect

Save & Connect

Análisis de requisitos

Contexto de Ecommify

Ecommify es un ecosistema tecnológico de comercio electrónico multiproveedor de alta escala diseñado bajo una arquitectura de Base de Datos Híbrida. La plataforma está concebida para conectar de forma eficiente a clientes distribuidos geográficamente con múltiples vendedores independientes, procesando un alto volumen de operaciones concurrentes por segundo.

Para resolver los problemas tradicionales de rendimiento y flexibilidad, la arquitectura de Ecommify fragmenta sus responsabilidades de almacenamiento en dos grandes frentes tecnológicos:

1. **Módulo Transaccional Relacional PostgreSQL:** Diseñado con un esquema rígido y normalizado. Se encarga de garantizar la consistencia en el procesamiento de órdenes, el desglose financiero de los pagos, las entregas de los pedidos gestionando rutas usando los códigos postales.
2. **Módulo Documental No Relacional MongoDB:** Diseñado con un esquema flexible, dinámico y desnormalizado. Se encarga del manejo de la alta demanda de lectura en el catálogo de productos, la gestión de campañas promocionales y el mantenimiento de las sesiones y carritos activos de los usuarios.

Requisitos funcionales

Los requisitos funcionales definen los servicios esenciales y las acciones específicas que la plataforma Ecommify debe ejecutar para satisfacer las reglas del negocio:

Módulo Transaccional y Logístico - Postgresql

- **Registro Unificado de Usuarios:** El sistema debe almacenar la identidad e información de Clientes y Vendedores, estandarizando la captura de sus ubicaciones mediante una estructura de dirección compuesta.
- **Ciclo de Vida de la Orden:** El sistema debe capturar el estado de cada pedido y registrar el periodo de tiempo exacto desde la orden de compra hasta la confirmación de la entrega física.
- **Cálculo de Rutas y Distancias:** La plataforma debe almacenar y procesar coordenadas geográficas precisas de los códigos postales para ejecutar consultas

de proximidad y estimar costos de flete basados en el origen del vendedor y destino del cliente.}

- **Gestión de productos:** Al realizar una venta, el sistema debe registrar el identificador de los artículos vendidos, el vendedor, el precio y el valor del flete asignado a cada producto.
- **Gestión de Pagos Múltiples:** El sistema debe permitir que una orden sea liquidada usando distintos métodos de pago, por ejemplo, tarjeta de crédito en cuotas más saldo de billetera digital, guardando el detalle de las transacciones financieras.

Módulo de Catálogo y Experiencia de Usuario - MongoDB

- **Gestión de Catálogo de producto:** El sistema debe permitir el almacenamiento de productos con características variables que dependen exclusivamente del tipo de categoría, sin requerir una estructura de columnas fija.
- **Gestión de Campañas Promocionales:** El catálogo debe asociar de manera las promociones activas a cada producto, especificando su porcentaje de descuento y fecha límite de vigencia.
- **Gestión de Galería Multimedia:** Cada producto debe tener una lista de referencias a recursos visuales como imágenes o videos para poder poner en las interfaces de usuario.
- **Gestión de la Calificación de Productos:** El sistema debe mantener un resumen precalculado de la valoración del producto con una calificación promedio y total de reseñas, este enfoque permite evitar procesar consultas de agregación durante las búsquedas que realizan los usuarios.
- **Gestión de Carritos Temporales:** La plataforma debe guardar en tiempo real los estados de los carritos de compras activos vinculados a una sesión, registrando los productos y cantidades seleccionadas por el usuario.

Requisitos no funcionales

Los requisitos no funcionales especifican las propiedades de calidad, restricciones técnicas y criterios de rendimiento que debe cumplir la infraestructura híbrida de Ecommify:

- **Cumplimiento Estricto de Propiedades ACID:** El motor relacional PostgreSQL debe asegurar atomicidad, consistencia, aislamiento y durabilidad en todas las operaciones de las órdenes y detalles financieros, asegurando que un cobro jamás sea procesado si la orden falla.

- **Latencia Mínima en Lecturas de Catálogo:** La recuperación de la ficha completa de un producto incluyendo fotos, especificaciones, promociones y reputación debe resolverse en una sola operación de lectura en MongoDB, garantizando tiempos de respuesta inferiores a 50 milisegundos.
- **Alta Disponibilidad y Tolerancia a Fallos:** El módulo que gestiona las sesiones de clientes y carritos de compra debe estar diseñado para tolerar caídas de nodos sin degradar la experiencia de navegación en picos masivos de tráfico.
- **Precisión Financiera:** Los campos que representan valores monetarios precios y fletes en el módulo transaccional deben almacenarse bajo tipos de datos numéricos exactos de alta precisión para eliminar los errores de redondeo asociados a tipos de punto flotante.
- **Eficiencia en Búsquedas sobre Estructuras Complejas:** Las propiedades semiestructuradas para especificaciones y pagos y las listas de datos para fotos deben optimizarse mediante índices GIN para agilizar el filtrado de datos.
- **Evolución de Esquema sin Interrupciones:** La colección del catálogo en MongoDB debe operar bajo el principio de esquema flexible, permitiendo añadir nuevas categorías o variar estructuras de atributos sin necesidad hacer cambios en la db.
- **Manejo de Ciclo de Vida de Sesiones:** Los documentos que registran la actividad de sesión del usuario deben admitir la configuración de políticas de expiración automática.

Identificación de entidades del dominio

El modelo de dominio de Ecommify se ha dividido de manera estratégica siguiendo el enfoque de Persistencia Polígota. A continuación, se describen y justifican las entidades que componen el sistema, clasificadas según el motor de base de datos asignado para su persistencia.

Entidades del Dominio Relacional (PostgreSQL)

Estas entidades representan los componentes core del negocio. Requieren estructuras estables, relaciones bien definidas mediante integridad referencial y un cumplimiento estricto de las propiedades ACID.

Cliente (ECOMMIFY_CUSTOMERS)

- Descripción: Representa al actor principal que realiza las compras en el ecosistema. Almacena la información de identidad e inicio de sesión.
- Atributos Clave: `customer_id` (Identificador único transaccional) y `customer_unique_id` (Identificador lógico de cuenta de usuario).
- Particularidad de Diseño: Incorpora el atributo compuesto `customer_address` (basado en el tipo definido por el usuario `address_type`), el cual encapsula de forma estructurada el prefijo postal, la ciudad y el estado, promoviendo la reutilización de código en la base de datos.

Vendedor (ECOMMIFY_SELLERS)

- Descripción: Modela a las tiendas o proveedores independientes que ofrecen sus artículos dentro del Marketplace.
- Atributos Clave: `seller_id` (Llave primaria).
- Particularidad de Diseño: Al igual que el cliente, implementa la estructura compuesta `seller_address`. Esto estandariza la captura del punto de procedencia o despacho de la mercancía, vital para los cálculos del motor logístico.

Orden (ECOMMIFY_ORDERS)

- Descripción: Es la entidad transaccional central del negocio. Documenta el acuerdo de compra formalizado por un cliente y realiza el seguimiento de su ciclo de vida operacional.
- Atributos Clave: `order_id` (Llave primaria) y `customer_id` (Llave foránea).
- Particularidad de Diseño: Utiliza un tipo de rango con zona horaria (`tstzrange`) denominado `order_logistics_timeline`. Este campo almacena de forma atómica y en una sola columna el momento exacto de la compra (`purchase_timestamp`) y el momento de la entrega (`delivered_timestamp`). Esto optimiza las consultas analíticas de tiempos de entrega sin requerir múltiples columnas independientes y asegurando que la fecha de entrega sea lógicamente posterior a la de compra.

Detalles de la Orden (ECOMMIFY_ORDER_DETAILS)

- Descripción: Entidad asociativa de granularidad fina que desglosa cada uno de los ítems contenidos en una orden de compra. Vincula los productos comprados con sus respectivos vendedores, congelando los precios y costos de flete del momento exacto de la transacción.
- Atributos Clave: `order_item_id` (Secuencial interno) en conjunto con `order_id`, `product_id` y `seller_id` como llaves foráneas.
- Particularidad de Diseño: Incorpora el campo `payment_details` estructurado como un tipo de dato `jsonb`. Este campo almacena de forma flexible las transacciones financieras (por ejemplo, si el usuario usó dos tarjetas de crédito distintas o combinó un cupón con efectivo), respaldado por un índice GIN para búsquedas instantáneas por método de pago.

Catálogo Maestro de Productos (ECOMMIFY_PRODUCTS)

- Descripción: Entidad que actúa como el registro maestro indexable de los artículos del inventario dentro del entorno relacional.
- Atributos Clave: product_id (Llave primaria).
- Particularidad de Diseño: Para convivir con la flexibilidad del catálogo, implementa el atributo product_specifications como jsonb (soportado por un índice GIN para buscar métricas o dimensiones físicas heterogéneas) y product_photos como un arreglo de texto (text_array), optimizando el almacenamiento de múltiples URL de imágenes en un solo registro.

Geolocalización (ECOMMIFY_GEOLOCATION)

- Descripción: Infraestructura geoespacial base del sistema que provee los puntos cartográficos de todas las regiones de operación.
- Atributos Clave: geolocation_zip_code_prefix (Llave primaria).
- Particularidad de Diseño: Define el atributo geolocation_point utilizando la extensión espacial PostGIS bajo el tipo geometry(Point, 4326). Esto permite al motor relacional calcular de forma nativa distancias vectoriales y optimizar rutas de despacho calculando la cercanía entre el prefijo postal del vendedor y el del cliente.

Entidades del Dominio Documental (MongoDB)

Estas entidades están diseñadas bajo el enfoque de esquema flexible y desnormalizado. Su objetivo principal es asegurar una alta disponibilidad y lecturas ultrarrápidas, eliminando los cuellos de botella generados por la alta concurrencia en la navegación del sitio.

Producto y Reseñas CATALOG_PRODUCTS_COLLECTION

- Descripción: Representa el documento unificado y autocontenido que se expone directamente en la interfaz de usuario web o móvil de cara al cliente final durante la navegación.
- Identificador Único: _id es un String mapeado desde el product_id lógico de PostgreSQL.
- Estructura del Documento:
 - category: Clasificación textual para indexación y filtrado en búsquedas.
 - active_promotions: Almacena las ofertas vigentes (is_promo, discount, valid_until) evitando consultas a colecciones externas de marketing.
 - attributes: Estructura en formato objeto clave-valor que permite almacenar de forma libre las dimensiones físicas.
 - Media (Arreglo): Lista indexada de strings con las URL de las fotografías del artículo.
 - reviews_summary es un subdocumento embebido: Guarda un acumulado precalculado de la reputación (avg_score y total_reviews). Este diseño evita que el sistema tenga que contar y promediar millones de registros de calificaciones en tiempo real cada vez que un cliente visualiza un producto.

Sesión y Carrito de Compra CUSTOMER_SESSIONS_COLLECTION

- Descripción: Encargada de mantener el estado de la sesión activa de un usuario y los artículos seleccionados antes de proceder a la compra formal.

- Identificador Único: `_id`.
- Estructura del Documento:
 - `customer_id`: Enlace hacia el cliente maestro en PostgreSQL.
 - `last_activity`: Timestamp modificado con cada acción.
 - `active_cart` (Subdocumento Embebido): Con el campo calculado total y un arreglo interno de elementos (`items`), donde cada registro guarda el identificador del producto (`prod_id`), la cantidad solicitada (`qty`) y el momento en que se añadió (`added_at`). Esto permite un flujo de adición y edición de ítems en el carrito.

Diseño conceptual

El siguiente diseño representa el motor transaccional, financiero y logístico de Ecommify implementado en PostgreSQL.

- El Corazón ACID del Negocio: Gestiona los procesos operativos críticos que exigen consistencia absoluta y dinero, el registro de clientes, vendedores, facturación de pedidos y la liquidación de pagos.
- Capacidades Empresariales Avanzadas: No es un modelo relacional tradicional; incorpora características modernas de PostgreSQL como el almacenamiento de datos semiestructurados en `jsonb`, listas de imágenes en `text_array`, tipos compuestos para direcciones `address_type` y rangos nativos de tiempo `tstzrange` para medir líneas de tiempo logísticas de entrega.
- Inteligencia Geoespacial: Integra la extensión PostGIS (ECOMMIFY_GEOLOCATION) para almacenar coordenadas físicas con el fin de calcular distancias y fletes reales de envío entre las bodegas de los proveedores y los hogares de los clientes.

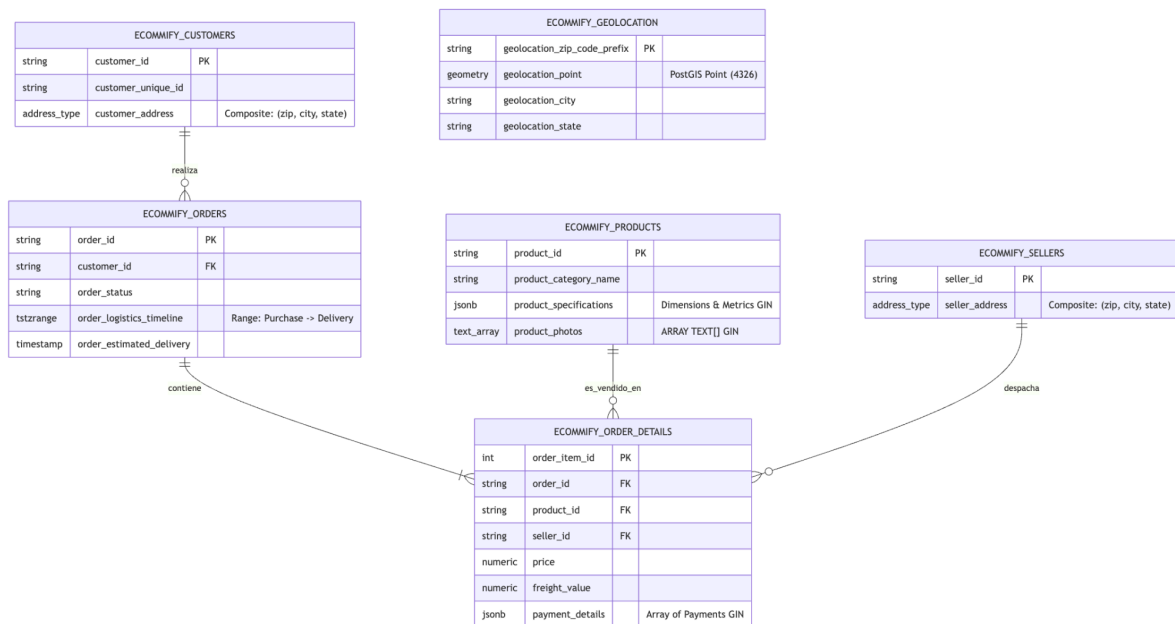
Garantiza que el dinero, las órdenes y las rutas de envío de la plataforma sean consistentes, precisos y nunca se dupliquen o pierdan.

Diagrama ER

Diseño SQL para PostgreSQL.

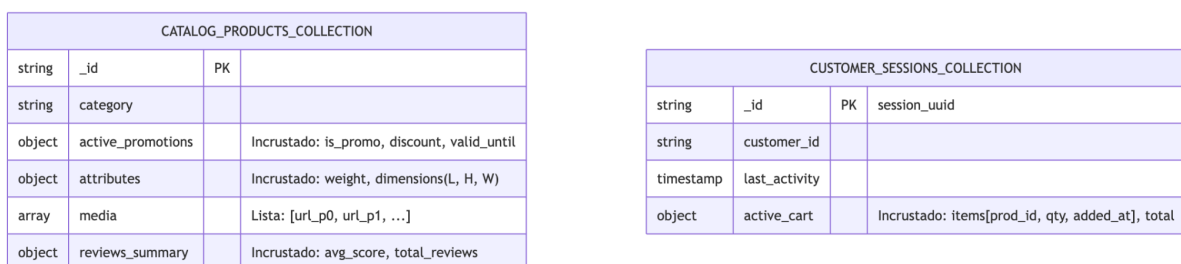
El diseño implementa una estructura normalizada de tercera forma normal para mitigar la redundancia de clientes, órdenes, ítems y vendedores, asegurando un estricto cumplimiento de las propiedades ACID en los procesos financieros y de facturación.

El modelo destaca por la incorporación de extensiones avanzadas de ingeniería de datos, tales como el soporte geoespacial de PostGIS para la optimización de rutas logísticas y el uso de estructuras semiestructuradas jsonb para dotar de flexibilidad al desglose de pagos y especificaciones sin romper el esquema relacional central.



Diseño NoSQL para MongoDB

Mientras PostgreSQL protege la integridad financiera, MongoDB asume el control del catálogo web masivo y de los sistemas de personalización. Su arquitectura orientada a documentos es ideal para lecturas ultrarrápidas con *queries de baja latencia* al evitar por completo operaciones de unión de datos.



Descripción detallada de cada entidad y relación

A continuación, se presenta la descripción de cada entidad, sus atributos y las relaciones de cardinalidad

Descripción Detallada de las Entidades en PostgreSQL

ECOMMIFY_CUSTOMERS (Clientes)

- Es la entidad maestra que almacena la información de identidad de los compradores de la plataforma.
- Atributos:
 - `customer_id`: Identificador único interno para el seguimiento transaccional de cada cliente.
 - `customer_unique_id`: Identificador único de cuenta o credencial de usuario, ideal para persistir sesiones.
 - `customer_address (address_type)`: Atributo compuesto que unifica código postal, ciudad y estado.

ECOMMIFY_SELLERS (Vendedores)

- Es la entidad que registra a los vendedores o tiendas independientes autorizadas para vender dentro del marketplace.
- Atributos:
 - `seller_id`: Identificador único del comercio o proveedor.
 - `seller_address (address_type)`: Atributo compuesto código postal, ciudad, estado.

ECOMMIFY_ORDERS (Ordenes)

- Es la entidad transaccional central que registra las órdenes de cada cliente.
- Atributos:
 - `order_id`: Código único de la orden de compra.
 - `customer_id`: Vinculación con el cliente que generó la compra.
 - `order_status`: Estado actual de la orden, por ejemplo: pending, shipped, delivered.
 - `order_logistics_timeline (tstzrange)`: Tipo de dato avanzado de rango de tiempo con zona horaria. Encapsula en una sola celda el momento de la compra hasta la entrega.
 - `order_estimated_delivery (timestamp)`: Fecha estimada de entrega calculada por el motor de negocio.

ECOMMIFY_PRODUCTS (Productos)

- Registro maestro de los artículos disponibles para la venta dentro del entorno relacional ACID.
- Atributos:
 - `product_id`: Código único identificador del producto.
 - `product_category_name`: Categoría principal del producto.
 - `product_specifications (jsonb)`: Almacenamiento semiestructurado para guardar especificaciones muy variables.
 - `product_photos (text_array)`: Arreglo de strings conteniendo las URLs de las imágenes.

ECOMMIFY_ORDER_DETAILS (Detalles de la Orden e Ítems)

- Entidad que desglosa de manera exacta cada artículo comprado.
- Atributos:
 - order_item_id: Identificador de cada ítem dentro de la orden.
 - order_id: Referencia a la orden a la que pertenece el ítem.
 - product_id: Enlace al producto adquirido.
 - seller_id: Identificador del proveedor específico que surte dicho ítem.
 - price: Precio exacto del producto en un tipo de dato de precisión financiera.
 - freight_value: Costo de envío calculado de forma individual para ese ítem.
 - payment_details (jsonb): Almacenamiento estructurado del desglose de pagos por ítem

ECOMMIFY_GEOLOCATION (Geolocalización Logística)

- Catálogo de referencia espacial para cálculos de distancias y costos de rutas. No posee llaves foráneas rígidas para evitar cuellos de botella de concurrencia al consultar millones de coordenadas de envíos.
- Atributos:
 - geolocation_zip_code_prefix: Código postal de clientes y vendedores.
 - geolocation_point (geometry): Coordenada espacial basada en la extensión espacial PostGIS Point.
 - geolocation_city (string): Nombre de la ciudad.
 - geolocation_state (string): Estado o provincia.

Descripción Detallada de las Entidades en MongoDB

Descripción técnica detallada del diseño del Módulo NoSQL (MongoDB)

Descripción Detallada de las Colecciones

En la base de datos documental de MongoDB se habla de colecciones, y los registros se denominan documentos. Ambas colecciones aplican el Patrón de Incrustación (Embedding) para auto contener la información y maximizar el rendimiento.

Colección de Catálogo de Productos

Representa el documento unificado y desnormalizado de los artículos disponibles en la plataforma. Está diseñado para alimentar la interfaz de usuario de forma inmediata.

- Atributos y Estructura:
 - _id (string / PK): Identificador único del producto en el catálogo mapeado de forma lógica con el product_id de PostgreSQL.
 - category (string): Categoría principal a la que pertenece el producto para facilitar búsquedas y filtrados rápidos.

- active_promotions (object - Incrustado): Subdocumento que contiene la información comercial de ofertas vigentes
 - is_promo, discount, valid_until.
 - Al estar embebido, la capa de aplicación renderiza los precios especiales de inmediato sin consultar otra base de datos
- Attributes (object - Incrustado): Estructura flexible tipo objeto clave-valor que encapsula las dimensiones del producto
 - weight
 - dimensions[L, H, W])
 - Este campo permite el polimorfismo, admitiendo que diferentes categorías tengan especificaciones técnicas totalmente distintas.
- media (array - Lista): Arreglo de cadenas de texto [url_p0, url_p1, ...] que almacena las rutas URL de las fotografías o videos del artículo.
- reviews_summary (object - Incrustado): Subdocumento que guarda el cálculo en caché de la reputación del artículo
 - avg_score, total_reviews
 - Actúa como un caché analítico para evitar ejecutar operaciones de agregación masivas cuando miles de usuarios navegan simultáneamente por el catálogo.

Colección de Sesiones de Clientes

Entidad encargada de gestionar los carritos de compra y la persistencia de la actividad del usuario en tiempo real antes de pasar a la fase de pago formal.

- Atributos y Estructura:
 - _id (string / PK): Identificador único de la sesión (session_uuid).
 - customer_id (string): Cadena de texto que sirve como referencia de acoplamiento lógico hacia el registro maestro del cliente ubicado en PostgreSQL.
 - last_activity (timestamp): Registra el último segundo de interacción del usuario.
 - active_cart (object - Incrustado): Subdocumento que modela el carrito de compras.
 - items [prod_id, qty, added_at]: Contiene una lista interna de artículos que registra qué productos se seleccionaron, sus cantidades y cuándo se agregaron, junto con un campo calculado global llamado total.

Descripción de Relaciones e Integración Híbrida

En la arquitectura orientada a documentos de MongoDB se evita por completo el uso de relaciones rígidas o llaves foráneas nativas para maximizar las lecturas ultrarrápidas y de baja latencia. En su lugar, el diagrama plantea dos tipos de relaciones estructurales y lógicas:

Relaciones por Incrustación (Relación 1:1 y 1:N Interna)

- **Mecánica:** Las entidades tradicionales independientes, como: Promociones, Atributos, Resúmenes de Reseñas y Líneas del Carrito se fusionan directamente dentro del documento contenedor padre como subdocumentos u objetos embebidos.
- **Propósito:** Transforma lo que en un modelo relacional requeriría 4 o 5 tablas diferentes unidas mediante operaciones de JOIN, en una única lectura directa por llave primaria (`_id`), reduciendo el consumo de memoria I/O y CPU al mínimo.

Relación Lógica Inter-Motor (Acoplamiento Débil con PostgreSQL)

- **Mecánica:** Campos como `customer_id` en la colección de sesiones y el `_id` de la colección de productos no poseen restricciones de integridad referencial física en MongoDB. Son llaves lógicas.
- **Flujo del Negocio:** Cuando un usuario decide realizar el pago de su carrito de compras, la capa de la aplicación de Ecommify lee el documento de la sesión de MongoDB e inyecta estos datos en las tablas transaccionales de PostgreSQL para procesar el cobro bajo garantías ACID estrictas.

Restricciones de negocio identificadas

1. Una orden debe pertenecer a un único cliente

Cada registro en `ECOMMIFY_ORDERS` debe estar asociado obligatoriamente a un `customer_id` válido. No pueden existir órdenes huérfanas.

2. La fecha de entrega debe ser posterior a la fecha de compra

El atributo `order_logistics_timeline` modela el rango entre compra y entrega, garantizando coherencia temporal en el proceso logístico.

3. Un pedido puede tener múltiples métodos de pago

La plataforma permite pagos combinados (tarjeta, billetera digital), por lo que una orden puede tener varios detalles financieros asociados.

4. Cada ítem de la orden debe estar asociado a un producto y a un vendedor

No se puede registrar un detalle de compra sin identificar qué producto fue vendido y qué vendedor lo suministró.

5. Los valores monetarios deben almacenarse con precisión exacta

Los precios y costos de envío no pueden manejarse con tipos flotantes debido al riesgo de errores financieros y redondeos.

6. No puede procesarse un pago si la orden falla

El sistema debe garantizar propiedades ACID para evitar inconsistencias financieras o duplicación de cobros.

7. Cada producto debe tener un identificador único

El `product_id` funciona como clave maestra tanto en PostgreSQL como en MongoDB, asegurando trazabilidad entre motores.

8. Las promociones deben tener fecha de vigencia

Una promoción solo puede considerarse activa hasta el valor definido en `valid_until`.

9. Los carritos de compra deben estar asociados a una sesión activa

Cada carrito depende de un `session_uuid` y debe registrar actividad reciente del usuario mediante `last_activity`.

10. Las sesiones deben expirar automáticamente

El sistema debe eliminar sesiones inactivas para optimizar recursos y mantener la consistencia operativa.

11. El catálogo debe soportar productos con atributos variables

No todas las categorías tienen las mismas características técnicas, por lo que el sistema debe aceptar estructuras dinámicas sin alterar el esquema principal.

12. Las coordenadas geográficas deben permitir cálculo de rutas y distancias

Los códigos postales de clientes y vendedores deben estar asociados a coordenadas válidas para calcular fletes y proximidad logística.

Diseño lógico - Módulo PostgreSQL

- Esquema relacional normalizado (mínimo 3FN).

Primera Forma Normal (1FN)

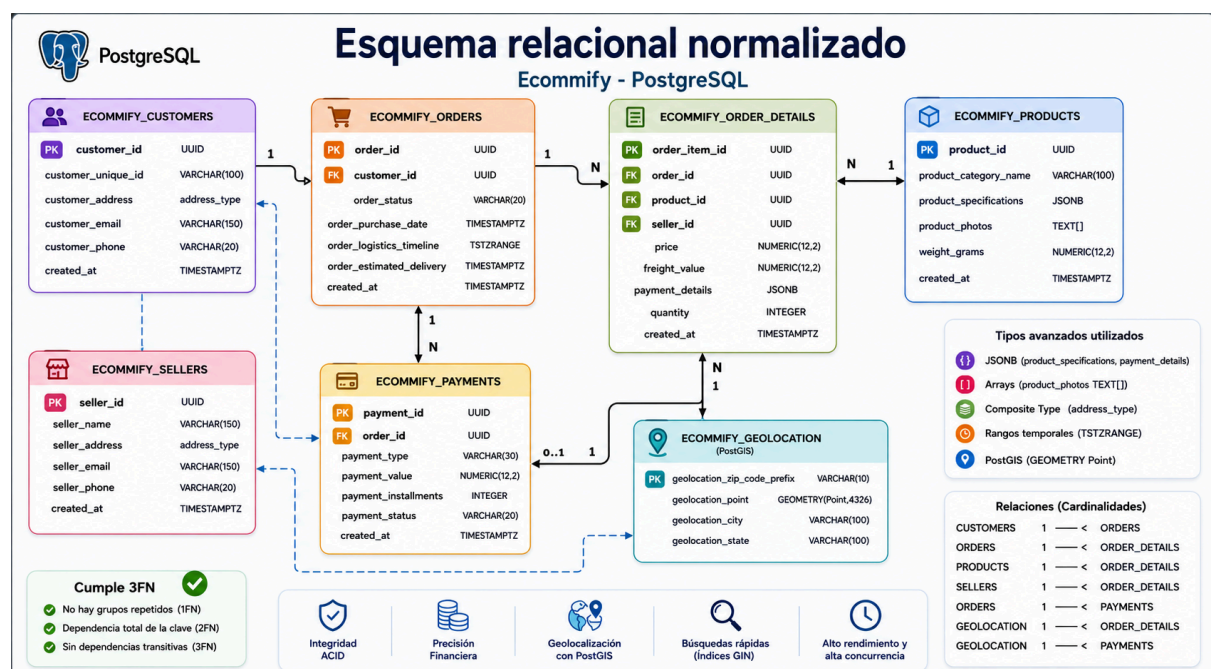
No hay atributos multivaluados repetidos en tablas transaccionales
Cada fila tiene PK única

Segunda Forma Normal (2FN)

Los atributos dependen completamente de la PK
Por ejemplo: price depende del item de la orden, no depende solo del order_id

Tercera Forma Normal (3FN)

No existen dependencias transitivas
Por ejemplo: ciudad y estado no se duplican en la tabla de órdenes
Se abstraen mediante address_type y geolocalización



- Tabla por tabla con columnas, tipos de datos, claves, constraints.

Tabla: ECOMMIFY_CUSTOMERS

Columna	Tipo	Constraint
customer_id	UUID	PK
customer_unique_id	VARCHAR(100)	UNIQUE NOT NULL
customer_address	address_type	NOT NULL

Tabla: ECOMMIFY_SELLERS

Columna	Tipo de dato	Constraint
seller_id	UUID	PRIMARY KEY
seller_address	address_type	NOT NULL

Tabla: ECOMMIFY_ORDERS

Columna	Tipo	Constraint
order_id	UUID	PK
customer_id	UUID	FK
order_status	VARCHAR(30)	CHECK
order_logistics_timeline	tstzrange	NOT NULL
order_estimated_delivery	TIMESTAMP	NULL

Tabla: ECOMMIFY_PRODUCTS

Columna	Tipo de dato	Constraint
product_id	UUID	PRIMARY KEY
product_category_name	VARCHAR(100)	NOT NULL
product_specifications	JSONB	NULL
product_photos	TEXT[]	NULL

Tabla: ECOMMIFY_ORDER_DETAILS

Columna	Tipo de dato	Constraint
order_item_id	BIGSERIAL	PRIMARY KEY
order_id	UUID	FOREIGN KEY NOT NULL
product_id	UUID	FOREIGN KEY NOT NULL
seller_id	UUID	FOREIGN KEY NOT NULL
price	NUMERIC(12,2)	NOT NULL
freight_value	NUMERIC(12,2)	NOT NULL
payment_details	JSONB	NULL

Tabla: ECOMMIFY_GEOLOCATION

Columna	Tipo de dato	Constraint
geolocation_zip_code_prefix	VARCHAR(10)	PRIMARY KEY
geolocation_point	geometry(Point,4326)	NOT NULL
geolocation_city	VARCHAR(100)	NOT NULL
geolocation_state	VARCHAR(100)	NOT NULL

- Aplicación de tipos avanzados (JSONB, arrays, composite types) justificada.

El tipo JSONB se utiliza en atributos como `product_specifications` y `payment_details`. En el caso de los productos, permite almacenar especificaciones variables según la categoría, evitando esquemas rígidos con múltiples columnas nulas. Para los pagos, facilita registrar transacciones combinadas con distintos métodos financieros dentro de una misma orden. Además, este tipo de dato puede optimizarse mediante índices GIN para acelerar búsquedas sobre estructuras semiestructuradas.

Los arreglos se implementan en el atributo `product_photos` para almacenar múltiples URLs de imágenes dentro de un único registro. Esta estrategia simplifica el modelo relacional y reduce la necesidad de tablas auxiliares o múltiples operaciones JOIN al consultar galerías multimedia de productos.

Por otra parte, el sistema implementa tipos compuestos mediante `address_type`, utilizado tanto en clientes como en vendedores. Este tipo encapsula código postal, ciudad, departamento y país en una sola estructura reusable, promoviendo consistencia semántica y reducción de redundancia en el modelo de datos.

PostgreSQL incorpora el tipo avanzado `TSTZRANGE` para modelar el ciclo de vida logístico de las órdenes, almacenando en un único atributo el rango temporal entre la compra y la entrega. Así mismo, la extensión PostGIS permite utilizar el tipo `geometry(Point,4326)` para soportar cálculos espaciales, distancias y optimización de rutas logísticas dentro de la plataforma.

- Extensiones a utilizar con justificación.

Se puede utilizar la extensión de **PostGIS** para poder calcular rutas, distancias y proximidad de puntos geográficos. También se puede trabajar con **GIN** para realizar las búsquedas sobre los objetos JSONB y como último se usa también **pgcrypto** para la generación aleatoria de los UUIDs

- Estrategia de particionamiento si aplica.

La mejor tabla para realizar el particionamiento es `ECOMMIFY_ORDERS` porque va a crecer muchísimo con cada orden que se agregue. Una estrategia puede ser particionarla por la fecha de compra (mes, año, etc)

Diseño lógico preliminar - Módulo MongoDB.

El módulo MongoDB de Ecommify se diseña para responder a las necesidades de consulta rápida, flexibilidad de estructura y alta disponibilidad en los componentes relacionados con la experiencia de usuario. A diferencia del módulo PostgreSQL, que se enfoca en la consistencia transaccional, MongoDB se orienta a escenarios donde la prioridad es entregar información completa del catálogo y del carrito con la menor latencia posible.

Colecciones principales

Para el módulo documental se proponen dos colecciones principales:

1. **catalog_products_collection**
Almacena la ficha completa de cada producto disponible en la plataforma. Incluye categoría, promociones activas, atributos variables, galería multimedia y resumen de reseñas.
2. **customer_sessions_collection**
Almacena las sesiones activas de los clientes y sus carritos de compra temporales.

Incluye identificador de sesión, cliente asociado, última actividad y productos agregados al carrito.

Estas colecciones se diseñan aplicando el patrón de documentos embebidos, con el fin de reducir operaciones de unión y permitir que la aplicación recupere la información necesaria en una sola consulta.

catalog_products_collection

Esta colección representa el catálogo de productos usado por la interfaz web o móvil de Ecommify. Su propósito es entregar al usuario toda la información necesaria para visualizar un producto sin consultar múltiples tablas o colecciones.

Estructura propuesta del documento

```
{
  "_id": "product_001",
  "category": "electronics",
  "product_name": "Smartphone X Pro",
  "active_promotions": {
    "is_promo": true,
    "discount": 15,
    "valid_until": "2026-06-30T23:59:59Z"
  },
  "attributes": {
    "brand": "EcommifyTech",
    "color": "black",
    "storage": "128GB",
    "weight_g": 180,
    "dimensions": {
      "length_cm": 15,
      "height_cm": 7,
      "width_cm": 1
    }
  },
  "media": [
    "https://ecommify.com/products/product_001/front.jpg",
    "https://ecommify.com/products/product_001/side.jpg",
    "https://ecommify.com/products/product_001/detail.jpg"
  ],
  "reviews_summary": {
    "avg_score": 4.7,
    "total_reviews": 186
  },
  "postgres_product_id": "product_001",
  "created_at": "2026-05-18T10:00:00Z",
  "updated_at": "2026-05-18T10:00:00Z"
}
```

La colección de productos se modela como un documento autocontenido porque la ficha de producto requiere información de diferentes naturalezas: categoría, atributos técnicos, imágenes, promociones y reseñas, en un modelo relacional tradicional, esta información podría estar distribuida en varias tablas, generando múltiples operaciones **JOIN**, en MongoDB, estos elementos se embeben dentro del mismo documento, permitiendo que la consulta por **_id** entregue toda la información necesaria para pintar la vista del producto.

El campo **attributes** se define como un objeto flexible, ya que cada categoría puede tener especificaciones diferentes. Por ejemplo, un celular puede tener memoria, color y capacidad de batería; mientras que una prenda puede tener talla, material y género. Este comportamiento justifica el uso de MongoDB para el catálogo, porque evita modificar el esquema cada vez que se agrega una nueva categoría o atributo.

customer_sessions_collection

Esta colección almacena las sesiones activas de los clientes y los carritos de compra temporales. Su diseño está orientado a disponibilidad y velocidad, ya que el carrito debe actualizarse constantemente mientras el usuario navega por la plataforma.

```
{
  "_id": "session_8f7a9c",
  "customer_id": "customer_123",
  "last_activity": "2026-05-18T15:45:00Z",
  "device": {
    "type": "mobile",
    "browser": "Chrome",
    "ip": "192.168.1.20"
  },
  "active_cart": {
    "items": [
      {
        "prod_id": "product_001",
        "qty": 2,
        "unit_price": 850000,
        "added_at": "2026-05-18T15:20:00Z"
      },
      {
        "prod_id": "product_008",
        "qty": 1,
        "unit_price": 120000,
        "added_at": "2026-05-18T15:30:00Z"
      }
    ],
    "total": 1820000
  },
  "status": "active",
}
```

```
"expires_at": "2026-05-19T15:45:00Z"
}
```

El carrito de compras se almacena como un subdocumento embebido dentro de la sesión porque sus elementos pertenecen directamente a la actividad temporal de un usuario, esta decisión evita crear una colección separada de carritos y otra de ítems del carrito, lo cual aumentaría la complejidad y el número de consultas necesarias.

El campo `last_activity` permite controlar la vigencia de la sesión, además, el campo `expires_at` puede utilizarse para configurar índices TTL en MongoDB, de manera que las sesiones antiguas se eliminan automáticamente, esta estrategia ayuda a optimizar almacenamiento y mantener limpia la base documental.

Patrones de modelado aplicados

1. Patrón de incrustación

Se utiliza para almacenar promociones, atributos, imágenes, reseñas resumidas e ítems del carrito dentro del documento principal, este patrón reduce la necesidad de consultas adicionales y mejora el rendimiento en escenarios de lectura intensiva.

2. Patrón de documento autocontenido

Cada producto del catálogo se diseña como una unidad completa de consulta, la aplicación puede recuperar una ficha de producto mediante una sola operación, lo cual mejora la experiencia del usuario y reduce la latencia.

3. Patrón de referencia lógica entre motores

Aunque MongoDB no utiliza llaves foráneas físicas hacia PostgreSQL, se mantiene una referencia lógica mediante campos como `postgres_product_id` y `customer_id`, estos campos permiten relacionar el catálogo y las sesiones con los datos maestros del motor transaccional.

4. Patrón de resumen precalculado

El campo `reviews_summary` guarda el promedio de calificación y el total de reseñas del producto, esta decisión evita calcular agregaciones pesadas en tiempo real cada vez que un usuario consulta el catálogo.

5. Patrón TTL para sesiones temporales

La colección de sesiones puede usar un índice TTL sobre el campo `expires_at`, permitiendo eliminar automáticamente documentos vencidos, esto es útil para carritos abandonados y sesiones inactivas.

Ejemplo de índice TTL:

```
db.customer_sessions_collection.createIndex(  
  { "expires_at": 1 },  
  { expireAfterSeconds: 0 }  
);
```

Índices recomendados en MongoDB

Para mejorar el rendimiento de las consultas, se proponen los siguientes índices:

```
db.catalog_products_collection.createIndex({ "category": 1 });  
  
db.catalog_products_collection.createIndex({ "active_promotions.is_promo": 1 });  
  
db.catalog_products_collection.createIndex({ "reviews_summary.avg_score": -1 });  
  
db.customer_sessions_collection.createIndex({ "customer_id": 1 });  
  
db.customer_sessions_collection.createIndex({ "last_activity": -1 });  
  
db.customer_sessions_collection.createIndex(  
  { "expires_at": 1 },  
  { expireAfterSeconds: 0 }  
);
```

Estos índices permiten acelerar búsquedas por categoría, productos promocionados, productos mejor calificados, sesiones por cliente y eliminación automática de sesiones vencidas.

Clasificación de qué datos van a MongoDB vs PostgreSQL

MongoDB se utiliza para datos que necesitan flexibilidad, baja latencia y lectura rápida. En este grupo se encuentran el catálogo de productos, promociones activas, atributos variables, imágenes, reseñas resumidas y carritos temporales, estos datos cambian con frecuencia en su estructura o se consultan muchas veces durante la navegación del usuario.

PostgreSQL se reserva para datos que requieren consistencia estricta, transacciones ACID, integridad referencial y precisión financiera, en este grupo se encuentran clientes, vendedores, órdenes, detalles de órdenes, pagos, precios de venta registrados y datos logísticos asociados a la entrega.

Esta separación permite que cada motor se use según sus fortalezas: PostgreSQL protege la integridad del negocio y MongoDB optimiza la experiencia de consulta del usuario.

Decisiones arquitectónicas justificadas

La arquitectura seleccionada para Ecommify es una arquitectura de persistencia políglota híbrida. Esta decisión se fundamenta en que la plataforma no maneja un único tipo de carga de trabajo, sino una combinación de procesos transaccionales, consultas analíticas, navegación de catálogo, sesiones temporales y operaciones logísticas.

Usar PostgreSQL para el núcleo transaccional

PostgreSQL se utiliza para almacenar órdenes, detalles de órdenes, clientes, vendedores, pagos y geolocalización logística, esta decisión se justifica porque estos datos requieren integridad, consistencia y control transaccional, una orden no puede quedar registrada sin cliente, un pago no puede procesarse si falla la orden y un detalle de compra no puede existir sin producto o vendedor asociado.

Además, PostgreSQL permite usar tipos avanzados como `JSONB`, `TSTZRANGE`, `TEXT[]`, tipos compuestos y extensiones como PostGIS. Esto permite que el modelo relacional no sea rígido, sino que pueda incorporar estructuras modernas sin perder consistencia.

Usar MongoDB para catálogo y experiencia de usuario

MongoDB se utiliza para el catálogo web, promociones, reseñas resumidas, sesiones y carritos de compra. Esta decisión se justifica porque estos datos requieren flexibilidad de esquema, disponibilidad y consultas rápidas.

En el catálogo, los productos pueden tener atributos diferentes según su categoría, por esta razón, un modelo documental es más adecuado que un modelo tabular rígido, además, al embeber promociones, imágenes y reseñas dentro del documento del producto, la aplicación puede consultar toda la ficha del producto en una sola operación.

Usar acoplamiento débil entre PostgreSQL y MongoDB

La integración entre ambos motores se realiza mediante referencias lógicas, no mediante restricciones físicas. Por ejemplo, el `_id` del producto en MongoDB coincide lógicamente con el `product_id` de PostgreSQL, y el `customer_id` de la sesión en MongoDB se relaciona con el cliente almacenado en PostgreSQL.

Esta decisión reduce la dependencia directa entre motores y permite que cada uno escale de forma independiente, sin embargo, exige que la aplicación controle la sincronización y valide la existencia de los datos antes de confirmar procesos críticos como el pago.

Usar documentos embebidos en MongoDB

Las promociones, atributos, medios, reseñas resumidas e ítems del carrito se modelan como subdocumentos embebidos, esta decisión se toma porque estos elementos se consultan casi siempre junto con su documento principal, por ejemplo, cuando un usuario visualiza un producto, necesita ver su categoría, imágenes, promoción y calificación al mismo tiempo.

El patrón de incrustación reduce operaciones adicionales y mejora el rendimiento de lectura, su principal limitación es que puede generar duplicidad o documentos grandes si se abusa del embebido, por lo que debe aplicarse solo a información directamente relacionada con el documento padre.

Usar resumen precalculado de reseñas

El campo `reviews_summary` almacena el promedio de calificación y el total de reseñas, esta decisión evita calcular promedios en tiempo real sobre grandes volúmenes de datos, en un comercio electrónico, miles de usuarios pueden consultar productos simultáneamente, por lo que precalcular estos valores mejora el rendimiento.

El costo de esta decisión es que el resumen puede quedar temporalmente desactualizado. Para mitigarlo, se puede actualizar mediante procesos programados o eventos cada vez que se registre una nueva reseña.

EDA del dataset Olist

El análisis exploratorio de datos, conocido como EDA, se desarrolló con el propósito de comprender la estructura, calidad, volumen y comportamiento del dataset Olist antes de tomar decisiones definitivas sobre el diseño de base de datos de Ecommify, este análisis permitió identificar las relaciones principales entre tablas, detectar valores faltantes, revisar distribuciones, observar patrones temporales y geográficos, y justificar técnicamente la separación de responsabilidades entre PostgreSQL y MongoDB.

El dataset de Olist se compone de varias tablas relacionadas con clientes, órdenes, productos, vendedores, pagos, reseñas y geolocalización, esta estructura refleja el funcionamiento de un marketplace real, donde una orden conecta al cliente con los productos adquiridos, los vendedores que los ofrecen, los métodos de pago utilizados y la información logística asociada a la entrega, en la hoja de ruta del proyecto se identificó que la tabla **orders** funciona como el centro del ecosistema, debido a que se conecta con clientes, ítems de orden, pagos y reseñas mediante claves como `order_id` y `customer_id`

Para el análisis se utilizó Google Colab como entorno principal, apoyado en Python y librerías como `pandas`, `matplotlib` y `seaborn`, en el script de trabajo se cargan los archivos CSV originales del dataset Olist, se recorren las tablas disponibles y se identifican columnas compartidas para detectar posibles relaciones entre entidades, además, el

archivo de integración genera conteos de registros por dataset, revisa valores nulos y transforma los datos originales en archivos preparados para PostgreSQL, como clientes, vendedores, productos, órdenes, geolocalización y detalles de orden, aca podemos ver la carga inicial de los datos:

```
import pandas as pd
import os
import matplotlib.pyplot as plt
import seaborn as sns

DATA_DIR = "/content/drive/MyDrive/Esteban/Estudio/Maestria
Arquitectura/Semestre I/Diseño y Optimización de Bases de Datos/Proyecto
Integrador - Ecommify/Kaggle_Dataset"

tables = {}

for file in os.listdir(DATA_DIR):
    if file.endswith(".csv"):
        name = file.replace("olist_", "").replace("_dataset.csv", "").replace(".csv", "")
        tables[name] = pd.read_csv(os.path.join(DATA_DIR, file))
        print(f"Tabla cargada: {name}")
```

Este proceso permitió centralizar las tablas en un diccionario de DataFrames y facilitar los análisis posteriores, además, permitió verificar que el dataset estaba compuesto por archivos relacionados entre sí, lo cual es importante para justificar un diseño relacional normalizado en el módulo transaccional.

Relaciones entre tablas

El análisis de columnas compartidas permitió identificar las relaciones principales entre los archivos, la tabla `orders` se relaciona con `customers` mediante `customer_id`; también se relaciona con `order_items`, `order_payments` y `order_reviews` mediante `order_id`, a su vez, `order_items` conecta con `products` mediante `product_id` y con `sellers` mediante `seller_id`.

Estas relaciones muestran que el dataset posee una estructura fuertemente transaccional, donde la integridad referencial es necesaria para evitar inconsistencias, por ejemplo, no debería existir un detalle de orden sin una orden válida, ni una orden sin un cliente asociado, esta característica justifica que el núcleo operativo de Ecommify sea modelado en PostgreSQL, debido a su capacidad para manejar llaves primarias, llaves foráneas, restricciones y transacciones ACID.

```
from collections import defaultdict
col_map = defaultdict(list)
```

```
for name, df in tables.items():
    for col in df.columns:
        col_map[col].append(name)

print("--- Posibles Relaciones Detectadas ---")
for col, tabs in col_map.items():
    if len(tabs) > 1:
        print(f"Columna '{col}' conecta las tablas: {tabs}")
```

Desde el punto de vista arquitectónico, este hallazgo confirma que las entidades de clientes, órdenes, detalles de orden, productos, vendedores, pagos y geolocalización deben mantener relaciones claras y controladas. Por eso, se recomienda que estos datos se conserven en PostgreSQL, mientras que la información orientada a experiencia de usuario, como catálogo enriquecido, promociones, multimedia, reseñas resumidas y carritos activos, se gestione en MongoDB.

Calidad de datos

El análisis de calidad de datos se enfocó en identificar valores faltantes, duplicados, tipos de datos incorrectos y posibles inconsistencias, en el archivo `olist_orders_dataset.csv` contiene varios registros y 8 columnas, inicialmente cargadas como tipo `object`, lo cual implica que las fechas deben convertirse explícitamente a tipo `datetime` antes de realizar análisis temporales o cálculos logísticos.

```
for nombre, df in tables.items():
    print(f"\n--- Datos faltantes en: {nombre} ---")
    nulos = df.isnull().sum()
    print(nulos[nulos > 0])
```

En la tabla de órdenes, los valores nulos se concentran principalmente en campos asociados al ciclo de vida logístico, como fechas de aprobación, entrega al transportador y entrega al cliente. Esto es esperable, debido a que algunas órdenes pueden estar canceladas, pendientes o no entregadas. Sin embargo, estos nulos deben ser considerados en el diseño de la base de datos, especialmente al usar el tipo `TSTZRANGE` para representar el periodo entre la compra y la entrega.

Para Ecommify, esto implica que el modelo debe permitir órdenes con estados diferentes y líneas de tiempo incompletas. Por ejemplo, una orden cancelada no necesariamente tendrá fecha de entrega. Por esa razón, el campo `order_logistics_timeline` debe construirse únicamente cuando existan fechas válidas, o manejar rangos abiertos en los casos donde la entrega aún no se ha completado.

Estadísticas básicas

Las estadísticas básicas permiten conocer el volumen general de cada tabla, la cantidad de registros y la estructura del dataset, este análisis resulta importante porque ayuda a estimar qué tablas pueden crecer más rápidamente en un entorno real de comercio electrónico.

```
conteo_datos = {}

for archivo in os.listdir(DATA_DIR):
    if archivo.endswith(".csv"):
        path_completo = os.path.join(DATA_DIR, archivo)
        try:
            df = pd.read_csv(path_completo, usecols=[0])
            conteo_datos[archivo] = len(df)
        except:
            print(f"Error al leer {archivo}")

conteo_ordenado = dict(sorted(conteo_datos.items(), key=lambda item: item[1]))

plt.figure(figsize=(12, 6))
plt.bar(conteo_ordenado.keys(), conteo_ordenado.values())
plt.xlabel("Datasets")
plt.ylabel("Número de registros")
plt.title("Comparativa: cantidad de datos por dataset")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

El conteo por archivo permite identificar que algunas tablas, como geolocalización, órdenes e ítems de orden, pueden tener un volumen considerable, este comportamiento es relevante para el diseño físico, porque indica dónde se deben priorizar índices, particionamiento y estrategias de optimización, en particular, la tabla de órdenes es candidata a particionamiento por fecha, debido a que crecerá de manera constante con cada compra registrada en la plataforma.

Distribución geográfica

En la operación logística de Ecommify tendrá una mayor carga en ciertas regiones, lo cual puede influir en la planificación de rutas, almacenamiento, vendedores cercanos y costos de envío:

```
customers_df = pd.read_csv(os.path.join(DATA_DIR,
"olist_customers_dataset.csv"))

plt.figure(figsize=(12, 6))
sns.countplot(
    data=customers_df,
    x="customer_state",
```

```
order=customers_df["customer_state"].value_counts().index
)
plt.title("Distribución geográfica de clientes por estado")
plt.xlabel("Estado")
plt.ylabel("Cantidad de clientes")
plt.show()
```

Este resultado justifica el uso de PostGIS en PostgreSQL, ya que las coordenadas de clientes y vendedores pueden convertirse en puntos geográficos para calcular distancias, estimar costos de flete y optimizar rutas, en la investigación previa se había definido que los datos de geolocalización pueden transformarse al tipo `GEOMETRY(Point, 4326)` para permitir cálculos espaciales indexados con GiST entre vendedor y cliente

Distribución temporal

El análisis temporal se realizó convirtiendo la columna `order_purchase_timestamp` a tipo `datetime` y agrupando las órdenes por mes, este análisis permite identificar picos de demanda, estacionalidad y comportamiento histórico de las compras.

```
orders_df = pd.read_csv(os.path.join(DATA_DIR, "olist_orders_dataset.csv"))

orders_df["order_purchase_timestamp"] = pd.to_datetime(
    orders_df["order_purchase_timestamp"]
)

orders_df.set_index("order_purchase_timestamp").resample("M").size().plot(
    figsize=(12, 6),
    marker="o"
)

plt.title("Evolución temporal de compras por mes")
plt.ylabel("Número de órdenes")
plt.grid(True)
plt.show()
```

Este patrón es importante porque evidencia que la arquitectura debe soportar picos de tráfico y mayor volumen de escrituras en determinados periodos, desde el diseño de base de datos, esto respalda el uso de particionamiento temporal en la tabla `ECOMMIFY_ORDERS`, así como la implementación de índices adecuados para consultas por fecha.

Valores nulos

Los valores nulos no deben interpretarse siempre como errores, en algunos casos representan eventos que no ocurrieron, como una orden que no fue entregada o una fecha

de aprobación ausente por cancelación, sin embargo, en campos críticos como identificadores, precios o llaves de relación, los nulos sí representarían un problema de integridad.

```
orders_nulls = orders_df.isnull().sum()
orders_nulls_pct = (orders_df.isnull().sum() / len(orders_df)) * 100

print("Valores nulos por columna:")
print(orders_nulls)

print("\nPorcentaje de valores nulos:")
print(orders_nulls_pct)
```

Para Ecommify, este análisis permite establecer reglas de negocio y restricciones, por ejemplo, `order_id`, `customer_id` y `order_status` deben ser obligatorios, en cambio, algunas fechas logísticas pueden ser opcionales dependiendo del estado de la orden, esta diferencia debe reflejarse en el diseño lógico de PostgreSQL, usando restricciones `NOT NULL` únicamente en campos realmente obligatorios.

Outliers

El análisis de outliers se aplica principalmente a variables numéricas como precio, valor de flete, peso y dimensiones del producto. Estos valores pueden revelar errores de carga, productos atípicos o casos de negocio especiales.

```
order_items = pd.read_csv(os.path.join(DATA_DIR,
"olist_order_items_dataset.csv"))

plt.figure(figsize=(8, 5))
plt.boxplot(order_items["price"].dropna())
plt.title("Outliers en precios")
plt.ylabel("Precio")
plt.show()

plt.figure(figsize=(8, 5))
plt.boxplot(order_items["freight_value"].dropna())
plt.title("Outliers en valores de flete")
plt.ylabel("Flete")
plt.show()
```

La presencia de valores extremos en precios o fletes debe ser revisada antes de diseñar dashboards o reglas automáticas de negocio, en una plataforma real, un flete demasiado alto podría afectar la experiencia del usuario, mientras que un precio atípico podría representar un producto especializado o un error de digitación, estos hallazgos justifican la necesidad de reglas de validación en la capa de aplicación y restricciones de precisión financiera en PostgreSQL mediante tipos `NUMERIC`.

Patrones del dataset

El EDA evidencia tres patrones principales. Primero, el dataset tiene un núcleo claramente transaccional, donde órdenes, clientes, pagos, productos y vendedores están fuertemente relacionados, segundo, existe una dimensión geográfica relevante, debido a la presencia de códigos postales, ciudades, estados, latitud y longitud, como tercer punto, existe información de catálogo que puede ser flexible, como atributos físicos de productos, imágenes, promociones o resúmenes de reseñas.

Estos patrones sustentan la arquitectura híbrida del proyecto. PostgreSQL resulta adecuado para la información transaccional y logística, mientras que MongoDB resulta más conveniente para la información de catálogo y experiencia de usuario. Esta separación coincide con la presentación ejecutiva del proyecto, donde se afirma que PostgreSQL protege el núcleo financiero y MongoDB habilita la experiencia masiva del usuario .

Matriz de decisiones

La siguiente matriz justifica qué información debe almacenarse en PostgreSQL y qué información debe almacenarse en MongoDB, los criterios utilizados son: estructura del dato, nivel de consistencia requerido, volumen esperado, tipo de consulta, frecuencia de acceso, necesidad de flexibilidad y criticidad del negocio.

Entidad o grupo de datos	Motor recomendado	Criterio principal	Justificación
Clientes	PostgreSQL	Consistencia e integridad	Los clientes se relacionan directamente con órdenes, pagos y direcciones, requieren identificadores únicos, trazabilidad e integridad referencial.
Vendedores	PostgreSQL	Integridad referencial	Los vendedores están asociados a los productos vendidos y al cálculo logístico del despacho, no deben existir ventas sin vendedor válido.
Órdenes	PostgreSQL	Transaccionalidad ACID	La orden representa el núcleo del negocio, debe garantizarse que no existan órdenes huérfanas ni estados inconsistentes.

Detalles de orden	PostgreSQL	Relación maestro-detalle	Cada ítem depende de una orden, un producto y un vendedor, requiere llaves foráneas y precisión financiera.
Pagos	PostgreSQL	Precisión y seguridad financiera	Los pagos no pueden duplicarse ni perderse, deben quedar asociados a una orden válida y manejarse bajo transacciones ACID.
Precios y fletes	PostgreSQL	Exactitud monetaria	Los valores monetarios deben almacenarse con tipos NUMERIC , evitando errores de redondeo.
Geolocalización	PostgreSQL + PostGIS	Cálculo espacial	Las coordenadas permiten calcular distancias entre vendedor y cliente, estimar costos de envío y optimizar rutas.
Catálogo web de productos	MongoDB	Lectura rápida y flexibilidad	El catálogo requiere atributos variables, multimedia, promociones y reseñas en una sola consulta.
Atributos variables del producto	MongoDB	Esquema flexible	Cada categoría puede tener atributos distintos, por lo que un documento flexible evita columnas nulas o cambios constantes de esquema.
Promociones activas	MongoDB	Baja latencia	Las promociones se consultan junto con el producto, al embeberlas, se evita consultar otra tabla o colección.
Galería multimedia	MongoDB	Documento autocontenido	Las imágenes y videos se consultan al visualizar el producto, es conveniente almacenarlos dentro del documento del catálogo.
Resumen de reseñas	MongoDB	Precalculo analítico	Guardar promedio y total de reseñas evita agregaciones costosas en tiempo real.

Sesiones de usuario	MongoDB	Alta disponibilidad	Las sesiones requieren lectura y escritura rápida, además de tolerancia a fallos.
Carritos temporales	MongoDB	Velocidad y expiración	El carrito cambia constantemente y puede eliminarse automáticamente mediante TTL si el usuario abandona la sesión.
Reportes históricos	PostgreSQL / vistas materializadas	Consulta analítica controlada	Los reportes de ventas pueden generarse desde PostgreSQL usando vistas materializadas y particionamiento por fecha.

Esta matriz responde al criterio solicitado en el entregable, donde se pide justificar PostgreSQL frente a MongoDB mediante criterios de volumen, estructura, consultas y consistencia

Análisis del Teorema CAP aplicado

El Teorema CAP establece que en un sistema distribuido no es posible garantizar simultáneamente consistencia fuerte, disponibilidad total y tolerancia a particiones, por esta razón, la arquitectura de Ecommify distribuye sus responsabilidades de acuerdo con la prioridad de cada módulo.

PostgreSQL: enfoque CP para el núcleo transaccional

El módulo PostgreSQL se orienta hacia la consistencia y la tolerancia a particiones controlada, en este módulo se almacenan órdenes, pagos, detalles de orden, clientes, vendedores y datos logísticos, la prioridad es que la información sea correcta y que las transacciones no queden en estados intermedios.

En una operación de compra, no sería aceptable que el sistema registre un pago sin crear la orden, o que descuente dinero sin asociar los productos comprados, por esta razón, PostgreSQL debe priorizar la consistencia sobre la disponibilidad absoluta, si existe un fallo durante una transacción crítica, es preferible rechazar o revertir la operación antes que confirmar datos inconsistentes.

Desde la perspectiva CAP, PostgreSQL se acerca a un enfoque CP, porque en los procesos financieros y transaccionales la consistencia es más importante que responder a toda costa.

MongoDB: enfoque AP para catálogo, sesiones y carritos

MongoDB se orienta hacia disponibilidad y tolerancia a particiones. En Ecommify, este motor se usa para catálogo, promociones, reseñas resumidas, sesiones y carritos, estos componentes hacen parte de la experiencia de navegación, donde es preferible que el usuario pueda seguir consultando productos aunque algunos datos tengan una pequeña latencia de actualización.

Por ejemplo, si una promoción tarda algunos segundos en sincronizarse, el impacto es menor que dejar sin respuesta el catálogo durante un pico de tráfico, del mismo modo, las sesiones y carritos requieren alta disponibilidad, porque el usuario espera que sus productos agregados se mantengan durante la navegación.

Desde la perspectiva CAP, MongoDB se aproxima a un enfoque AP, porque prioriza que el sistema siga disponible y tolerante a fallos, aceptando consistencia eventual en datos no financieros.

Aplicación en Ecommify

Módulo	Motor	Enfoque CAP	Justificación
Órdenes	PostgreSQL	CP	Se requiere consistencia estricta para evitar órdenes inválidas.
Pagos	PostgreSQL	CP	No se puede permitir duplicidad, pérdida o inconsistencia financiera.
Detalles de orden	PostgreSQL	CP	Cada detalle debe estar asociado a una orden, producto y vendedor.
Geolocalización logística	PostgreSQL + PostGIS	CP / CA controlado	Los cálculos deben ser confiables para estimar rutas y fletes.
Catálogo web	MongoDB	AP	Se prioriza disponibilidad y baja latencia en navegación.
Promociones	MongoDB	AP	Puede existir consistencia eventual sin afectar el núcleo financiero.
Sesiones	MongoDB	AP	Deben mantenerse disponibles durante picos de tráfico.
Carritos	MongoDB	AP	Se prioriza experiencia de usuario y rapidez de actualización.

La presentación ejecutiva del proyecto resume esta decisión indicando que PostgreSQL se enfoca en consistencia inmediata, mientras que MongoDB se orienta a disponibilidad, porque es preferible mostrar datos ligeramente latentes a que el sitio deje de responder.

Trade-offs y estrategias de mitigación

El uso de una arquitectura híbrida ofrece ventajas importantes, pero también introduce compromisos técnicos. A continuación, se describen los principales trade-offs y sus estrategias de mitigación.

Decisión	Beneficio	Riesgo	Estrategia de mitigación
Usar PostgreSQL para transacciones	Garantiza integridad, ACID y control financiero	Puede ser menos flexible para datos variables	Usar JSONB, tipos compuestos y rangos cuando sea necesario.
Usar MongoDB para catálogo	Permite baja latencia y documentos flexibles	Puede generar duplicidad frente al catálogo maestro	Sincronizar cambios desde PostgreSQL mediante eventos o procesos ETL.
Embeber promociones en productos	Reduce consultas y mejora tiempos de carga	Promociones podrían quedar temporalmente desactualizadas	Actualizar mediante jobs programados o eventos de cambio.
Guardar reviews_summary	Evita agregaciones masivas en tiempo real	El resumen puede no reflejar la última reseña inmediatamente	Recalcular el resumen cuando se registre una nueva reseña.
Manejar carritos en MongoDB	Mejora velocidad y disponibilidad	Riesgo de pérdida si la sesión expira antes del pago	Configurar TTL controlado y validar carrito antes del checkout.
Usar acoplamiento débil entre motores	Permite escalar PostgreSQL y MongoDB por separado	No hay integridad referencial física entre motores	Validar referencias en la capa de aplicación antes de confirmar compras.
Usar PostGIS para logística	Permite cálculo real de distancia y rutas	Puede aumentar costo computacional en consultas espaciales	Crear índices GiST y limitar cálculos a zonas relevantes.
Particionar órdenes por fecha	Mejora consultas históricas y mantenimiento	Aumenta complejidad administrativa	Automatizar creación de particiones mensuales.

Estos compromisos son coherentes con los desafíos presentados en la versión ejecutiva, donde se identifican como limitaciones la duplicidad de datos entre SQL y NoSQL, la complejidad de integridad inter-motor y la latencia en cálculos geoespaciales, proponiendo mitigaciones como colas de mensajes, patrón Saga, índices GiST y particionamiento por zonas postales

Flujos de sincronización entre PostgreSQL y MongoDB

La arquitectura híbrida de Ecommify requiere una estrategia de sincronización entre PostgreSQL y MongoDB. Esta sincronización no se basa en llaves foráneas físicas entre motores, sino en identificadores lógicos compartidos, como `product_id` y `customer_id`.

Flujo 1: Creación de producto

1. El producto se registra primero en PostgreSQL como dato maestro.
2. Se validan los campos obligatorios, como identificador, categoría y estado.
3. La aplicación genera un documento enriquecido para MongoDB.
4. MongoDB almacena la ficha del producto con atributos variables, imágenes, promociones y resumen de reseñas.
5. El producto queda disponible para consulta en el catálogo web.

Este flujo permite que PostgreSQL conserve la trazabilidad del producto y que MongoDB entregue una ficha completa de consulta rápida.

Flujo 2: Actualización de catálogo

1. Se actualiza un producto, promoción o atributo desde el sistema administrativo.
2. PostgreSQL conserva el cambio maestro si afecta información transaccional.
3. MongoDB actualiza el documento de catálogo si el cambio afecta la experiencia de usuario.
4. El usuario consulta el producto actualizado desde MongoDB.

La sincronización puede realizarse mediante servicios de aplicación, tareas programadas o mecanismos de Change Data Capture.

Flujo 3: Navegación del usuario

1. El usuario ingresa al catálogo de Ecommify.
2. La aplicación consulta MongoDB.
3. MongoDB retorna el producto completo con imágenes, promociones, atributos y reseñas resumidas.

4. No se consulta PostgreSQL durante la navegación normal.

Este flujo reduce la carga sobre el motor transaccional y mejora la latencia del portal.

Flujo 4: Carrito de compras

1. El usuario agrega un producto al carrito.
2. MongoDB actualiza el documento de sesión.
3. El carrito almacena producto, cantidad, precio visible y fecha de adición.
4. El documento se mantiene activo mientras exista actividad reciente.
5. Si la sesión expira, MongoDB elimina el documento mediante una política TTL.

Flujo 5: Checkout y confirmación de compra

1. El usuario decide pagar.
2. La aplicación lee el carrito activo desde MongoDB.
3. Antes de confirmar la venta, la aplicación valida en PostgreSQL que los productos, precios, vendedor y cliente sean válidos.
4. PostgreSQL inicia una transacción ACID.
5. Se crean la orden, los detalles de orden y los pagos.
6. Si todo es correcto, se confirma la transacción.
7. Si ocurre un error, se revierte la operación.
8. Finalmente, el carrito se elimina o se marca como finalizado en MongoDB.

Este flujo garantiza que MongoDB apoye la experiencia de usuario, pero que la información financiera final ocurra únicamente en PostgreSQL.

Anexos

Anexo A. Diccionario de datos preliminar PostgreSQL

Tabla	Campo	Tipo de dato	Descripción
ECOMMIFY_CUSTOMERS	customer_id	UUID	Identificador único del cliente.
ECOMMIFY_CUSTOMERS	customer_unique_id	VARCHAR(100)	Identificador lógico de cuenta.
ECOMMIFY_CUSTOMERS	customer_address	address_type	Dirección compuesta del cliente.

ECOMMIFY_SELLERS	seller_id	UUID	Identificador único del vendedor.
ECOMMIFY_SELLERS	seller_address	address_type	Dirección compuesta del vendedor.
ECOMMIFY_ORDERS	order_id	UUID	Identificador único de la orden.
ECOMMIFY_ORDERS	customer_id	UUID	Cliente que realizó la orden.
ECOMMIFY_ORDERS	order_status	VARCHAR(30)	Estado actual de la orden.
ECOMMIFY_ORDERS	order_logistics_timeline	TSTZRANGE	Rango entre compra y entrega.
ECOMMIFY_ORDERS	order_estimated_delivery	TIMESTAMP	Fecha estimada de entrega.
ECOMMIFY_PRODUCTS	product_id	UUID	Identificador único del producto.
ECOMMIFY_PRODUCTS	product_category_name	VARCHAR(100)	Categoría del producto.
ECOMMIFY_PRODUCTS	product_specifications	JSONB	Especificaciones variables del producto.
ECOMMIFY_PRODUCTS	product_photos	TEXT[]	Lista de URLs de imágenes.
ECOMMIFY_ORDER_DETAILS	order_item_id	BIGSERIAL	Identificador del ítem de orden.
ECOMMIFY_ORDER_DETAILS	order_id	UUID	Orden asociada.

ECOMMIFY_ORDER_DETAILS	product_id	UUID	Producto vendido.
ECOMMIFY_ORDER_DETAILS	seller_id	UUID	Vendedor asociado.
ECOMMIFY_ORDER_DETAILS	price	NUMERIC(12,2)	Precio exacto del producto.
ECOMMIFY_ORDER_DETAILS	freight_value	NUMERIC(12,2)	Valor exacto del flete.
ECOMMIFY_ORDER_DETAILS	payment_details	JSONB	Detalle flexible de pagos.
ECOMMIFY_GEOLOCATION	geolocation_zip_code_prefix	VARCHAR(10)	Código postal.
ECOMMIFY_GEOLOCATION	geolocation_point	geometry(Point,4326)	Coordenada geográfica.
ECOMMIFY_GEOLOCATION	geolocation_city	VARCHAR(100)	Ciudad.
ECOMMIFY_GEOLOCATION	geolocation_state	VARCHAR(100)	Estado.

Anexo B. Diccionario de datos preliminar MongoDB

Colección	Campo	Tipo	Descripción
catalog_products_collection	_id	string	Identificador del producto.
catalog_products_collection	category	string	Categoría del producto.
catalog_products_collection	product_name	string	Nombre comercial del producto.
catalog_products_collection	active_promotions	object	Promoción activa embebida.
catalog_products_collection	active_promotions.is_promo	boolean	Indica si el producto tiene promoción.

catalog_products_collection	active_promotions.discount	number	Porcentaje de descuento.
catalog_products_collection	active_promotions.valid_until	datetime	Fecha de vigencia.
catalog_products_collection	attributes	object	Atributos variables del producto.
catalog_products_collection	media	array	Lista de imágenes o videos.
catalog_products_collection	reviews_summary	object	Promedio y total de reseñas.
customer_sessions_collection	_id	string	Identificador de sesión.
customer_sessions_collection	customer_id	string	Referencia lógica al cliente.
customer_sessions_collection	last_activity	datetime	Última actividad del usuario.
customer_sessions_collection	active_cart	object	Carrito activo embebido.
customer_sessions_collection	active_cart.items	array	Productos agregados al carrito.
customer_sessions_collection	active_cart.total	number	Total calculado del carrito.
customer_sessions_collection	expires_at	datetime	Fecha de expiración automática.

Anexo C. Script preliminar MongoDB

```

use DataBaseOlistKaggle;

db.createCollection("catalog_products_collection");

db.createCollection("customer_sessions_collection");

db.catalog_products_collection.createIndex({ "category": 1 });

db.catalog_products_collection.createIndex({ "active_promotions.is_promo": 1 });

```



```
db.catalog_products_collection.createIndex({ "reviews_summary.avg_score": -1 });

db.customer_sessions_collection.createIndex({ "customer_id": 1 });

db.customer_sessions_collection.createIndex({ "last_activity": -1 });

db.customer_sessions_collection.createIndex(
  { "expires_at": 1 },
  { expireAfterSeconds: 0 }
);
```

Anexo D. Ejemplo de documento de producto en MongoDB

```
{
  "_id": "product_001",
  "category": "electronics",
  "product_name": "Smartphone X Pro",
  "active_promotions": {
    "is_promo": true,
    "discount": 15,
    "valid_until": "2026-06-30T23:59:59Z"
  },
  "attributes": {
    "brand": "EcommifyTech",
    "color": "black",
    "storage": "128GB",
    "weight_g": 180,
    "dimensions": {
      "length_cm": 15,
      "height_cm": 7,
      "width_cm": 1
    }
  },
  "media": [
    "https://ecommify.com/products/product_001/front.jpg",
    "https://ecommify.com/products/product_001/side.jpg"
  ],
  "reviews_summary": {
    "avg_score": 4.7,
    "total_reviews": 186
  },
  "postgres_product_id": "product_001",
  "created_at": "2026-05-18T10:00:00Z",
  "updated_at": "2026-05-18T10:00:00Z"
}
```

Anexo E. Ejemplo de documento de sesión y carrito

```
{
  "_id": "session_8f7a9c",
  "customer_id": "customer_123",
  "last_activity": "2026-05-18T15:45:00Z",
  "active_cart": {
    "items": [
      {
        "prod_id": "product_001",
        "qty": 2,
        "unit_price": 850000,
        "added_at": "2026-05-18T15:20:00Z"
      },
      {
        "prod_id": "product_008",
        "qty": 1,
        "unit_price": 120000,
        "added_at": "2026-05-18T15:30:00Z"
      }
    ],
    "total": 1820000
  },
  "status": "active",
  "expires_at": "2026-05-19T15:45:00Z"
}
```

Presentación ejecutiva

Diapositiva en:

<https://docs.google.com/presentation/d/12ek1805EnE1qOxnfLrHzrhB4M5IZjqKJ4wZ-LK5P1go/edit?usp=sharing>

https://drive.google.com/file/d/122T1Mg9FfeeTQdnbxzjwrgeF6oozNs-/view?usp=drive_link

Repositorio GitHub con código preliminar

El repositorio con la estructura planteada se encuentra en esta URL:

https://github.com/Admicail/Ecommify_Database_Desing

Referencias

- PostgreSQL Global Development Group. (2025). Data types. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/datatype.html>
- PostgreSQL Global Development Group. (2025). Server programming: Extending SQL. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/extend.html>